

# Programming in C

## Lab 6: Functions and Array

Lecturer: *Dr. Wan-Lei Zhao*  
*Spring Semester 2022*

## 1 Functions

## 2 Arrays

# Narcissus number

- Work out Narcissus number of 3 digits (100 ~ 999)
- 3 digits number satisfies:  $153 = 1^3 + 5^3 + 3^3$ 
  - ① function should be defined as '`int isNarcNum(int a)`'
  - ② Call it in main function to check numbers from 100 to 999
  - ③ Print out all Narcissus number in this range
  - ④ Two numbers in each line

# Print out Palindrome numbers

- Define a function `?? ispdl(int n)`
- Judge whether number `n` is a palindrome number
- Define a function `?? isqr(int n)`
- Judge whether number `n` is a square number
- Call this function in the `main()` function
- To print out all the Palindrome numbers in a given range `[a, b]`
- For example, 0, 1, 4, 9, 121, 484, 676, 10201, 12321
- Requirements
  - 1 Function `ispdl(int n)` returns `1` if `n` is a palindrome number
  - 2 Otherwise return `0`
  - 3 You should allow user to input `a` and `b`
  - 4 If `a ≥ b` or either of them is negative, output “invalid input!”

# Convert Number String to Integer: the interface

```
1 ?? ispld(int n)
2 {
3     //filling your code here
4 }
5
6 ?? isqr(int n)
7 {
8     //filling your code here
9 }
10
11 int main()
12 {
13     int i = 0;
14     for(i = 0; i < 10000; i++)
15     {
16         //filling your code here
17     }
18     return 0;
19 }
```

# Outline

1 Functions

2 Arrays

# Convert Number String to Integer: the problem

- Given a string `char a[]="312"`
- Convert it to number **312**
- Put it as a function `int str2num(char a[])`
- General steps:
  - 1 Calculate the length of the string
  - 2 From lower bit to higher bit do
  - 3 Convert each bit to number
  - 4 Add each bit up
  - 5 End-loop
- Hints
  - 1 Call `int strlen(char a[])` in `<string.h>`
  - 2 Example: `sz = strlen(a);`

# Convert Number String to Integer: the interface

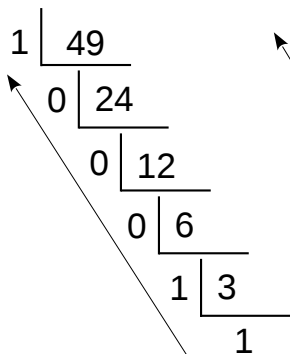
```
1 int str2num(??)
2 {
3     //filling your code here
4 }
5
6
7 int main()
8 {
9     char str[]="215";
10    int num = 0;
11    num = str2num(str);
12    printf("Num-is: %d\n", num);
13    return 0;
14 }
```

# Convert Decimal to its Hexadecimal: the problem

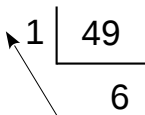
- Given an integer: 361005
- Convert the number to its hexadecimal: 0x5822D
- Put it as a function `void dec2hexa(int n)`
- General steps:
  - 1 Divide integer n with 16 recursively
  - 2 Keeps the modular in each time
  - 3 End-loop
  - 4 Map the resulting modulars to characters '0'~'9','A'~'F'
  - 5 Print the string inside `void dec2hexa(int n)`

# A Friendly Reminder: decimal to hexadecimal

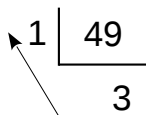
## Decimal to binary



## Decimal to octal



## Decimal to hexadecimal



10 → A  
11 → B  
12 → C  
13 → D  
14 → E  
15 → F

- Hints:** define a string `char hmap[] = "0123456789ABCDEF"`